

Competitive Programming

Section 26: Tree DP Basics

Competitive Programming Course

2026-07-06

Table of contents

1	Why tree DP comes after interval DP	3
2	Rooting a tree	4
3	Recognition	5
4	Pattern 1: include/exclude node	6
5	Pattern 2: counting colorings	7
6	Pattern 3: matching edges	8
7	Worked example: include/exclude	9
8	Worked example: colorings	10
9	C++ implementation notes	11
10	Python implementation notes	12
11	Proof habits	13
12	Common mistakes	14
13	Local exercises	15
13.1	A. Festival Invite	15
13.2	B. Road Pairing	15
13.3	C. Tree Colorings	15
14	Local judge workflow	16
15	Submission checklist	17

1 Why tree DP comes after interval DP

A tree has no cycles. Once you choose a root, every edge points from parent to child, and every subtree becomes an independent region after the parent's state is fixed.

Tree DP is the habit of asking:

If I know what happens at node u , what choices remain independently inside each child subtree

This section stays with one-root, bottom-up DP. Rerooting and LCA appear later.

2 Rooting a tree

The input tree is undirected. To do DP, pick any root, usually vertex 1.

After rooting:

```
parent[u] = previous vertex on the path to the root
children[u] = neighbors except parent[u]
subtree[u] = u plus all descendants of u
```

For large trees, avoid recursive DFS unless you deliberately handle stack limits. The local solutions build an iterative order:

```
order = [root]
for u in order:
    for v in adj[u]:
        if v has no parent:
            parent[v] = u
            append v to order

process reversed(order)
```

`reversed(order)` is postorder: every child is processed before its parent.

3 Recognition

Use tree DP when:

Statement signal	Likely state
choose vertices with parent-child conflict	include/exclude node
count colorings with adjacent restrictions	state by color of node
choose disjoint edges	matched/unmatched state
combine independent child subtrees	product or sum over children
answer depends on every root choice	future rerooting, not this section

4 Pattern 1: include/exclude node

For maximum independent set on a weighted tree:

```
take[u] = best total in subtree[u] if u is chosen
skip[u] = best total in subtree[u] if u is not chosen
```

If u is chosen, every child must be skipped:

```
take[u] = weight[u] + sum(skip[v])
```

If u is skipped, every child may be chosen or skipped:

```
skip[u] = sum(max(take[v], skip[v]))
```

Answer:

```
max(take[root], skip[root])
```

5 Pattern 2: counting colorings

For black/white colorings with no black-black edge:

```
white[u] = number of valid colorings of subtree[u] if u is white
black[u] = number of valid colorings of subtree[u] if u is black
```

If u is white, each child can be either color:

```
white[u] = product(white[v] + black[v])
```

If u is black, every child must be white:

```
black[u] = product(white[v])
```

Use modulo after each multiplication.

6 Pattern 3: matching edges

For maximum matching in a tree, each vertex can be incident to at most one chosen edge.

A useful state:

```
free[u] = best matching in subtree[u] when u is not matched to any child
used[u] = best matching in subtree[u] when u may be matched to one child
```

First compute the base where u is not matched to a child:

```
base = sum(max(free[v], used[v])) over children v
free[u] = base
```

Then try matching u to one child v :

```
candidate = base
            - max(free[v], used[v])
            + free[v]
            + 1
```

Why `free[v]`? If edge (u,v) is chosen, vertex v cannot be matched to one of its children.

7 Worked example: include/exclude

Tree:

```
1(5)
  2(1)
  3(4)
    4(6)
    5(3)
```

Leaves:

```
take[2]=1, skip[2]=0
take[4]=6, skip[4]=0
take[5]=3, skip[5]=0
```

Node 3:

```
take[3] = 4 + 0 + 0 = 4
skip[3] = max(6,0) + max(3,0) = 9
```

Root 1:

```
take[1] = 5 + skip[2] + skip[3] = 14
skip[1] = max(1,0) + max(4,9) = 10
```

Answer: 14.

8 Worked example: colorings

For a path of three vertices:

1 - 2 - 3

Valid colorings are all $2^3 = 8$ colorings except those where adjacent black vertices appear:

BBB, BBW, WBB

So the answer is 5.

The DP gives the same result at the root.

9 C++ implementation notes

Use adjacency lists:

```
vector<vector<int>>> adj(n);
```

Build an iterative parent/order array. Process **order** backward. Use **long long** for weighted sums and modulo products.

10 Python implementation notes

Use `sys.stdin.buffer.read()` for large trees. Iterative traversal avoids recursion limits:

```
parent = [-1] * n
order = [0]
parent[0] = 0
for u in order:
    ...
for u in reversed(order):
    ...
```

11 Proof habits

Tree DP proofs use induction on subtree height.

Template:

```
Assume every child state is correct.  
Given u's state, child subtrees are independent.  
The transition combines exactly the allowed choices for children.  
Therefore u's state is correct.
```

Independence is the key tree property.

12 Common mistakes

```
Processing parent before children.  
Forgetting to exclude the parent from children.  
Using recursion and crashing on a chain.  
For include/exclude, allowing a chosen child under a chosen parent.  
For matching, matching u to more than one child.  
For coloring counts, adding child choices when they should multiply.
```

13 Local exercises

13.1 A. Festival Invite

Weighted independent set:

```
take[u], skip[u]
```

13.2 B. Road Pairing

Maximum tree matching:

```
free[u], used[u]
```

13.3 C. Tree Colorings

Count valid black/white colorings:

```
white[u], black[u]
```

14 Local judge workflow

Run:

```
CP_TARGET=solution python3 sections/26_tree_dp_basics/check.py
```

Run everything:

```
CP_TARGET=solution python3 tools/check_all.py
```


15 Submission checklist

```
Did I root the tree?  
Do I know parent and children for every node?  
Are children processed before parents?  
What exactly does each state mean?  
Do child contributions sum, multiply, or take max?  
Do I need modulo or long long?
```